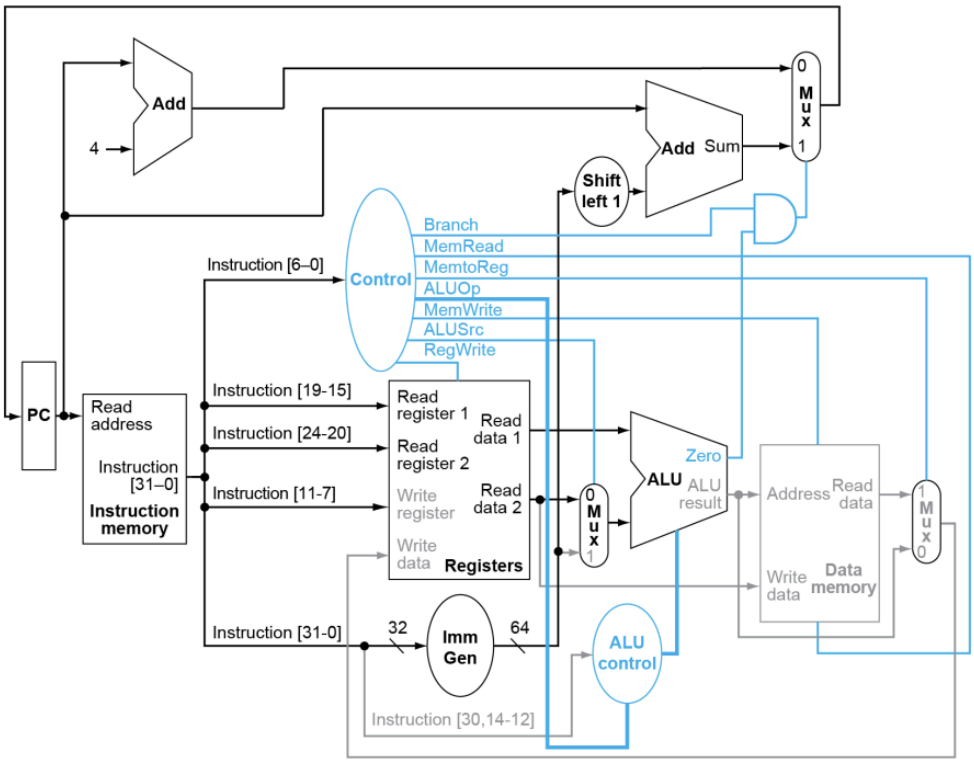
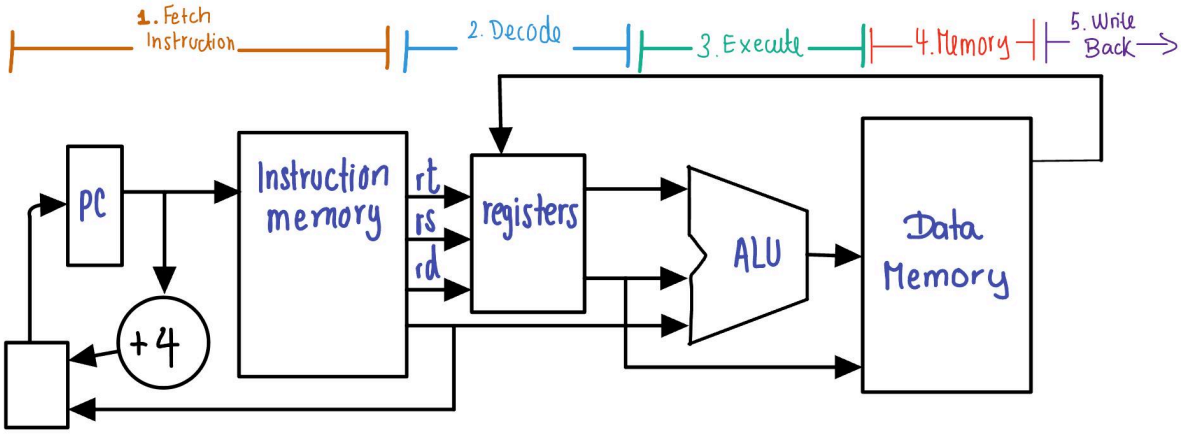
CSA - ECE 6913 - RISC V Processor Phase 2 Report

1) Draw the schematic for a single stage processor and fill in your code in the to run the simulator. (20 points)



This picture is from page 275 from the textbook and it's FIGURE 4.19 The datapath of Figure 4.11 with all necessary multiplexors and all control lines identified.
The following figure is one that I made to show how the single stage processor would work



All 5 stages complete in one clock cycle

1. (IF) Fetch instruction

Read instruction from Imem file → Using PC as the address → 32 bit instruction is fetched (Big Endian format) → PC = PC +4
My code shows this workflow stage in the following code snippet:

```
        PC = self.state.IF["PC"]
        instruction_binary = self.ext_imem.readInstr(PC)
```

2. (ID) Decode

Extract the opcode → read registers, imm → Control Signals are generated → read register file
My code shows this workflow stage in the following code snippet:

```
        instr = int(instruction_binary, 2)
        opcode = instr & 0x7F
        rd = (instr >> 7) & 0x1F
        funct3 = (instr >> 12) & 0x7
        rs1 = (instr >> 15) & 0x1F
        rs2 = (instr >> 20) & 0x1F
        funct7 = (instr >> 25) & 0x7F
```

3. (EX) Execute

ALU operations → SUB, ADD, XOR, OR AND → address calculation and branch comparison
My code shows this workflow stage in the following code snippet:

```
       # ---------------- R-type Instructions ----------------
        if opcode == 0x33:
            val1 = self.myRF.readRF(rs1)
            val2 = self.myRF.readRF(rs2)

            if funct3 == 0x0 and funct7 == 0x00:      # ADD
                result = (val1 + val2) & 0xFFFFFFFF
            elif funct3 == 0x0 and funct7 == 0x20:    # SUB
                result = (val1 - val2) & 0xFFFFFFFF
            elif funct3 == 0x4:                       # XOR
                result = val1 ^ val2
            elif funct3 == 0x6:                       # OR
                result = val1 | val2
            elif funct3 == 0x7:                       # AND
                result = val1 & val2
            else:
                result = 0
[.....more code.....]

        # ---------------- I-type Arithmetic ----------------
```

```
        elif opcode == 0x13:
            imm = (instr >> 20) & 0xFFF
            if imm & 0x800:  # Sign extend 12-bit
                imm |= 0xFFFFF000
```

4. (MEM) Memory

Read from DMEM (Load) and Write to DMEM (Read) → byte addressable → Big Endian format

My code shows this workflow stage in the following code snippet:

```
# ---------------- LOAD ----------------
        elif opcode == 0x03:
            imm = (instr >> 20) & 0xFFF
            if imm & 0x800:
                imm |= 0xFFFFF000
            addr = (self.myRF.readRF(rs1) + imm) & 0xFFFFFFFF
            data_binary = self.ext_dmem.readInstr(addr)
            data = int(data_binary, 2)
            self.myRF.writeRF(rd, data)
            self.nextState.IF["PC"] = PC + 4
```

```
# ---------------- STORE ----------------
        elif opcode == 0x23:
            imm = (((instr >> 25) & 0x7F) << 5) | ((instr >> 7) & 0x1F)
            if imm & 0x800:
                imm |= 0xFFFFF000
            addr = (self.myRF.readRF(rs1) + imm) & 0xFFFFFFFF
            data = self.myRF.readRF(rs2)
            self.ext_dmem.writeDataMem(addr, data)
            self.nextState.IF["PC"] = PC + 4
```

5. (WB) Write Back

Data source is selected by the MUX → Memory data/ALU result → Write to register file → r0 always equals 0.

My code shows this workflow stage in the following code snippet:

```
if opcode in [0x33, 0x13]:  # R-type or I-type
   self.myRF.writeRF(rd, result)
elif opcode == 0x03:  # LOAD
   self.myRF.writeRF(rd, data)
```

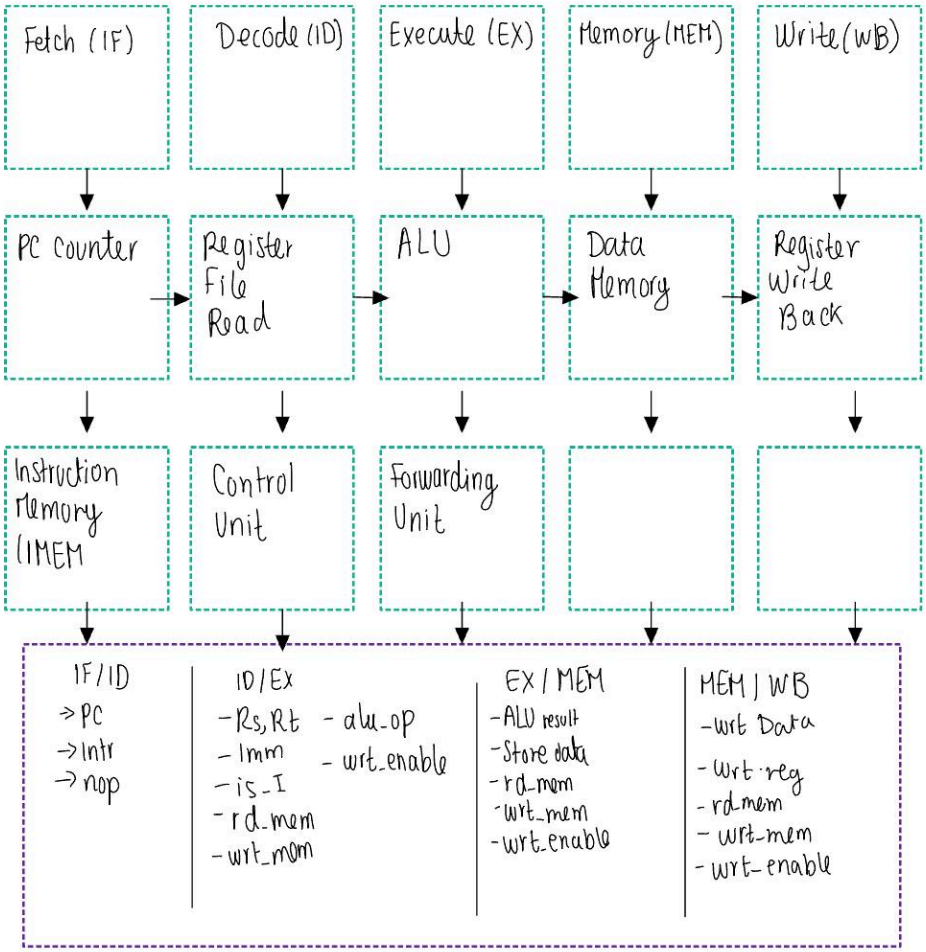
6. PC calculation

Move to next one
My code shows this workflow stage in the following code snippet:

```
self.nextState.IF["PC"] = PC + 4
```

2) Draw the schematic for a five stage pipelined processor and fill in your code to run the simulator. The processor should be able to take care of RAW and control hazards by stalling and forwarding. (20 points)

5-STAGE PIPELINE SCHEMATIC



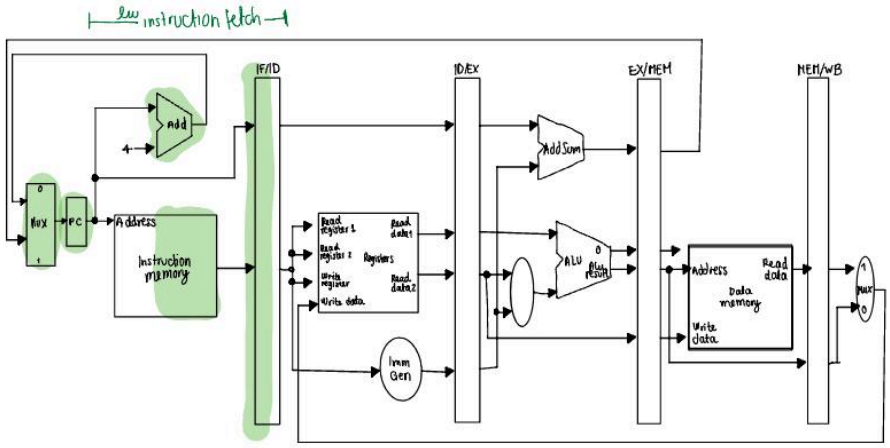
Implementation of Hazard Detection Unit

Forwarding Implementation
EX-EX
MEM-EX

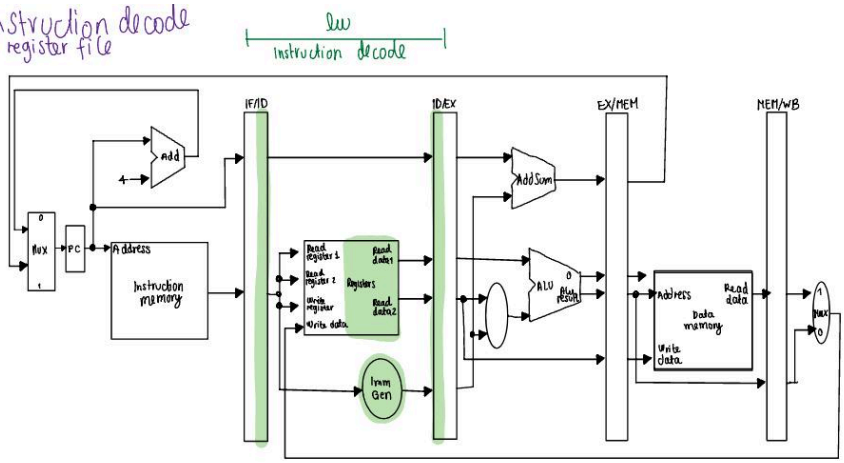
Branch Prediction
→ ID

I used the info from the book to outline the five-staged pipeline

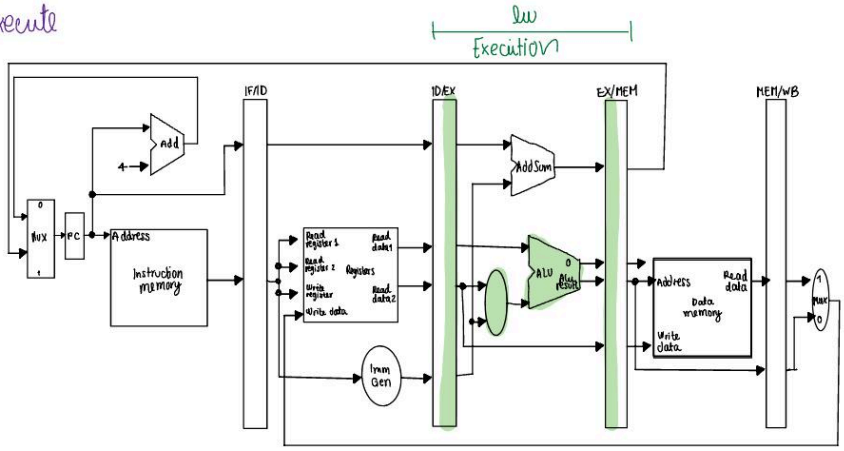
1. Instruction fetch

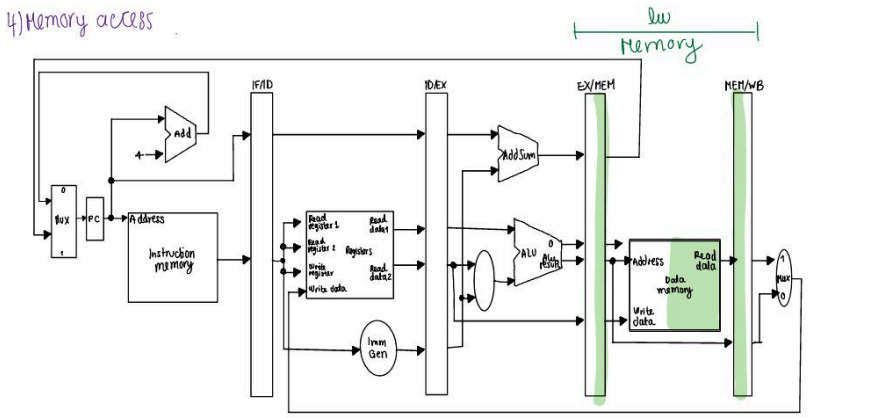


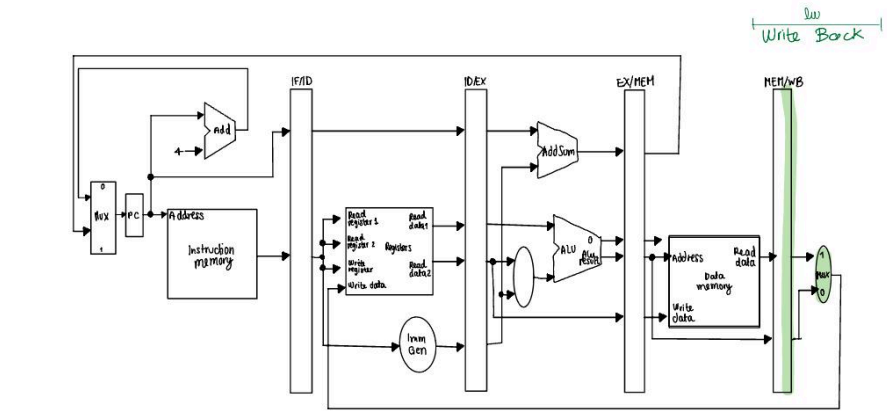
2. Instruction decode and register file



3. Execute







1. IF (Instruction Fetch) Stage:

```
# In IF stage:
PC = self.state.IF["PC"]
binary_instruction = self.ext_imem.readInstr(PC)
# Pass to ID stage
self.nextState.ID["Instr"] = instr
self.nextState.ID["nop"] = False
self.nextState.ID["PC"] = PC
self.nextState.IF["PC"] = PC + 4 # Next PC
```

2. ID (Instruction Decode) Stage: Decodes opcode (funct3, funct7) → Reads register file (RF) and detects hazards:

```
# Load-use hazard detection
if (not self.state.EX["nop"] and
   self.state.EX["rd_mem"] == 1 and
   self.state.EX["wrt_enable"] == 1):
```

```
  # Stall pipeline
   stall_IF_ID = True
   stall_ID_EX = True
# Resolves branches (in ID for early resolution):
if opcode == 0x63: # Branch
   taken = (val_rs1 == val_rs2) # for BEQ
   if taken:
       branch_taken = True
       branch_target = target_pc
       flush_IF_ID = True # Flush incorrectly fetched instr
```

3. EX (Execute) Stage: ALU operations (ADD, SUB, XOR, OR, AND), forwarding implemented

```
# Forwarding from MEM stage
if (not self.state.MEM["nop"] and
   self.state.MEM["wrt_enable"] == 1):
   if self.state.MEM["Wrt_reg_addr"] == self.state.EX["Rs"]:
       operand1 = self.state.MEM["ALUresult"] # Forward!
```

4. MEM (Memory Access) Stage: It forwards results to WB

Loads: if rd_mem == 1: read from data memory
Stores: if wrt_mem == 1: write to data memory

5. WB (Write Back) Stage:

Writes results to register file (I added some data hazard Implementation):

```
# Forwarding from MEM to EX
if (not self.state.MEM["nop"] and
   self.state.MEM["wrt_enable"] == 1 and
   self.state.MEM["Wrt_reg_addr"] != 0):
  # Forward to EX stage operands
   if self.state.MEM["Wrt_reg_addr"] == self.state.EX["Rs"]:
       operand1 = self.state.MEM["ALUresult"]

# Load-Use Hazards - Solved by Stalling:
# If EX has a load and ID needs that result
if (not self.state.EX["nop"] and
   self.state.EX["rd_mem"] == 1 and
   self.state.EX["wrt_enable"] == 1):
  # Stall for 1 cycle
```

```
stall_IF_ID = True # Freeze IF/ID
stall_ID_EX = True # Insert bubble in EX
```

C*3) Measure and report average CPI, Total execution cycles, and Instructions per cycle for both these cores by adding performance monitors to your code. (Submit code and print results to console or a file.) (5 points)*

The actual code is in the zip file in the code folder. I printed my results in the kernel and stored it as a file for each test as performancemetrics_ss. I wasn't sure what or how to include this so I just included everything. This is all under the submissions folder. But once I ran the test cases I got this:

For testcase 0 no debug statements:

SINGLE-STAGE CORE PERFORMANCE METRICS

==========

Total Execution Cycles: 7
Total Instructions Executed: 6
Average CPI (Cycles Per Instruction): 1.1667
IPC (Instructions Per Cycle): 0.8571

==========

FIVE-STAGE CORE PERFORMANCE METRICS

==========

Total Execution Cycles: 11
Total Instructions Executed: 6
Average CPI (Cycles Per Instruction): 1.8333
IPC (Instructions Per Cycle): 0.5455

==========

For testcase 1 no debug statements:

SINGLE-STAGE CORE PERFORMANCE METRICS

==========

Total Execution Cycles: 40
Total Instructions Executed: 39
Average CPI (Cycles Per Instruction): 1.0256
IPC (Instructions Per Cycle): 0.9750

==========

FIVE-STAGE CORE PERFORMANCE METRICS

==========

Total Execution Cycles: 46
Total Instructions Executed: 39
Average CPI (Cycles Per Instruction): 1.1795
IPC (Instructions Per Cycle): 0.8478

==========

For testcase 2 with debug statements:

[CYCLE 4] WB counting instruction: wrt_enable=1, rd_mem=1, wrt_mem=0, reg=2
[CYCLE 5] WB counting instruction: wrt_enable=1, rd_mem=1, wrt_mem=0, reg=3
[CYCLE 5] ID: BEQ at PC=16, rs1=3(21), rs2=5(0), taken=True, target=24
[CYCLE 6] WB counting instruction: wrt_enable=1, rd_mem=0, wrt_mem=0, reg=4
[CYCLE 7] WB counting instruction: wrt_enable=1, rd_mem=0, wrt_mem=0, reg=5
[CYCLE 7] ID: BEQ at PC=24, rs1=2(7), rs2=4(1), taken=True, target=8
[CYCLE 8] WB counting instruction: wrt_enable=0, rd_mem=0, wrt_mem=2, reg=0
[CYCLE 10] WB counting instruction: wrt_enable=0, rd_mem=0, wrt_mem=2, reg=0
[CYCLE 11] ID: BEQ at PC=16, rs1=3(21), rs2=5(1), taken=True, target=24
[CYCLE 12] WB counting instruction: wrt_enable=1, rd_mem=0, wrt_mem=0, reg=4
[CYCLE 13] WB counting instruction: wrt_enable=1, rd_mem=0, wrt_mem=0, reg=5
[CYCLE 13] ID: BEQ at PC=24, rs1=2(7), rs2=4(2), taken=True, target=8
[CYCLE 14] WB counting instruction: wrt_enable=0, rd_mem=0, wrt_mem=2, reg=0
[CYCLE 16] WB counting instruction: wrt_enable=0, rd_mem=0, wrt_mem=2, reg=0
[CYCLE 17] ID: BEQ at PC=16, rs1=3(21), rs2=5(3), taken=True, target=24
[CYCLE 18] WB counting instruction: wrt_enable=1, rd_mem=0, wrt_mem=0, reg=4
[CYCLE 19] WB counting instruction: wrt_enable=1, rd_mem=0, wrt_mem=0, reg=5

[CYCLE 19] ID: BEQ at PC=24, rs1=2(7), rs2=4(3), taken=True, target=8
[CYCLE 20] WB counting instruction: wrt_enable=0, rd_mem=0, wrt_mem=2, reg=0
[CYCLE 22] WB counting instruction: wrt_enable=0, rd_mem=0, wrt_mem=2, reg=0
[CYCLE 23] ID: BEQ at PC=16, rs1=3(21), rs2=5(6), taken=True, target=24
[CYCLE 24] WB counting instruction: wrt_enable=1, rd_mem=0, wrt_mem=0, reg=4
[CYCLE 25] WB counting instruction: wrt_enable=1, rd_mem=0, wrt_mem=0, reg=5
[CYCLE 25] ID: BEQ at PC=24, rs1=2(7), rs2=4(4), taken=True, target=8
[CYCLE 26] WB counting instruction: wrt_enable=0, rd_mem=0, wrt_mem=2, reg=0
[CYCLE 28] WB counting instruction: wrt_enable=0, rd_mem=0, wrt_mem=2, reg=0
[CYCLE 29] ID: BEQ at PC=16, rs1=3(21), rs2=5(10), taken=True, target=24
[CYCLE 30] WB counting instruction: wrt_enable=1, rd_mem=0, wrt_mem=0, reg=4
[CYCLE 31] WB counting instruction: wrt_enable=1, rd_mem=0, wrt_mem=0, reg=5
[CYCLE 31] ID: BEQ at PC=24, rs1=2(7), rs2=4(5), taken=True, target=8
[CYCLE 32] WB counting instruction: wrt_enable=0, rd_mem=0, wrt_mem=2, reg=0
[CYCLE 34] WB counting instruction: wrt_enable=0, rd_mem=0, wrt_mem=2, reg=0
[CYCLE 35] ID: BEQ at PC=16, rs1=3(21), rs2=5(15), taken=True, target=24
[CYCLE 36] WB counting instruction: wrt_enable=1, rd_mem=0, wrt_mem=0, reg=4
[CYCLE 37] WB counting instruction: wrt_enable=1, rd_mem=0, wrt_mem=0, reg=5
[CYCLE 37] ID: BEQ at PC=24, rs1=2(7), rs2=4(6), taken=True, target=8
[CYCLE 38] WB counting instruction: wrt_enable=0, rd_mem=0, wrt_mem=2, reg=0
[CYCLE 40] WB counting instruction: wrt_enable=0, rd_mem=0, wrt_mem=2, reg=0
[CYCLE 41] ID: BEQ at PC=16, rs1=3(21), rs2=5(21), taken=False, target=N/A
[CYCLE 42] WB counting instruction: wrt_enable=1, rd_mem=0, wrt_mem=0, reg=4
[CYCLE 42] ID: JAL at PC=20, target=32, return_addr=24
[CYCLE 43] WB counting instruction: wrt_enable=1, rd_mem=0, wrt_mem=0, reg=5
[CYCLE 44] WB counting instruction: wrt_enable=0, rd_mem=0, wrt_mem=2, reg=0
[CYCLE 45] WB counting instruction: wrt_enable=1, rd_mem=0, wrt_mem=0, reg=10
[CYCLE 46] ID: BEQ at PC=40, rs1=0(0), rs2=0(0), taken=True, target=24
[CYCLE 47] WB counting instruction: wrt_enable=0, rd_mem=0, wrt_mem=1, reg=0
[CYCLE 48] WB counting instruction: wrt_enable=0, rd_mem=0, wrt_mem=1, reg=0
[CYCLE 48] ID: BEQ at PC=24, rs1=2(7), rs2=4(7), taken=False, target=N/A
[CYCLE 49] WB counting instruction: wrt_enable=0, rd_mem=0, wrt_mem=2, reg=0
[CYCLE 51] WB counting instruction: wrt_enable=0, rd_mem=0, wrt_mem=2, reg=0
[CYCLE 52] WB counting instruction: wrt_enable=0, rd_mem=0, wrt_mem=3, reg=0
SINGLE-STAGE CORE PERFORMANCE METRICS
==========

Total Execution Cycles: 36
Total Instructions Executed: 35
Average CPI (Cycles Per Instruction): 1.0286
IPC (Instructions Per Cycle): 0.9722

==========

FIVE-STAGE CORE PERFORMANCE METRICS
==========
Total Execution Cycles: 53
Total Instructions Executed: 35
Average CPI (Cycles Per Instruction): 1.5143
IPC (Instructions Per Cycle): 0.6604
==========

For testcase 2 no debug statements:

SINGLE-STAGE CORE PERFORMANCE METRICS
==========
Total Execution Cycles: 36
Total Instructions Executed: 35
Average CPI (Cycles Per Instruction): 1.0286
IPC (Instructions Per Cycle): 0.9722
==========

FIVE-STAGE CORE PERFORMANCE METRICS
==========
Total Execution Cycles: 53
Total Instructions Executed: 35
Average CPI (Cycles Per Instruction): 1.5143
IPC (Instructions Per Cycle): 0.6604
==========

This result looks a bit different from the correct output metrics that were given to us in the output folder. I tried to make it reach 50 cycles but it went over because of stalls and data hazards since everytime I tried to modify them it would be affecting the other test cases.

4) Compare the results from both the single stage and the five stage pipelined processor implementations and explain why one is better than the other. (5 points)

From the test results it looks like the five-stage performs worse on smaller programs due to the overhead it causes. For testcase 1, we see that it takes 40 cycles , 39 instructions for the single one (CPI of 1.0256), while the five-stage was 46 cycles (CPI of 1.1795), causing the five stage pipeline to be worse than the single stage one due to the stalls caused by the branches. Finally for testcase 2, it's evident that the single stage is way better than the five stage which almost doubles

the cycles from single-stage (50 compared to 36 cycles)due to jumping through different stages. Therefore, we can say that single-stage is better for single-stage programs, with no hazards and has a very predictable timing, taking one cycle per instruction. Five-stage should usually have a higher clock frequency and usually for larger programs but in our case it just performs worse because of data hazard, stalls, overhead and forwarding. Also, each branch taken can cause 1 cycle stall.

5) What optimizations or features can be added to improve performance? (Extra credit 1 point)

I could apply forwarding for loads, or reorder instructions to avoid hazards, have a better system for branch prediction such as implementing a 1-bit or 2 bit predictor like this:

```
if self.branch_predict[pred_index] == TAKEN:
  # Prefetch coming from target being predicted
   self.nextState.IF["PC"] = predicted_tgt
else:
   self.nextState.IF["PC"] = PC + 4
```

Other solutions that could be harder or less useful for the test case given could be executing multiple instructions per cycle, since it might require improving and probably duplicating the hardware. Finally, we could execute instructions as soon as the operands become free to use and available, requiring reordering the buffer and reservation stations.

My simulator deals with branch instructions as follows:

1. Branches are always assumed to be NOT TAKEN. That is, when a new is fetched in the IF stage, the PC is speculatively updated as PC+4.
2. Branch conditions are resolved in the ID/RF stage.
3. If the branch is determined to be not taken in the ID/RF stage (as predicted), then the pipeline proceeds without disruptions. If the branch is determined to be taken, then the speculatively fetched instruction is discarded and the nop bit is set for the ID/RR stage for the next cycle. Then the new instruction is fetched in the next cycle using the new branch PC address.